Plan / Build 模式 —— 实际例子端到端全流程解析

> 以「为项目添加 OAuth2 登录功能」为实例，结合源码逐行剖析 Prometheus → Atlas → Sisyphus-Junior 的完整 Plan / Build 流程。

目录

1. 场景概述与角色清单

用户输入：`/plan 为项目添加 OAuth2 登录功能，支持 Google 和 GitHub 两个 Provider`

参与角色

角色	Agent	职责	可写文件
规划者	Prometheus	面试、研究、生成计划	`.sisyphus/*.md` 仅
编排者	Atlas	读取计划、委派任务、验证结果	`.sisyphus/notepads/` + 验证命令

角色	Agent	职责	可写文件
执行者	Sisyphus-Junior	实际写代码、跑测试	任意文件
研究者	Explore / Librarian	代码搜索、文档查阅	无（只读）
顾问	Metis	缺口分析	无（只读）
审计者	Momus （可选高精度模式）	计划审核	无（只读）
决策者	Oracle	架构决策	无（只读）

2. Phase 0 — Agent 注册与配置组装

当 OpenCode 加载 oh-my-opencode 插件时， `config` hook 被调用。在 `applyAgentConfig()` 中完成 Prometheus 和 Plan agent 的注册：

2.1 Prometheus Agent 配置构建

> 源文件：`src/plugin-handlers/agent-config-handler.ts` (236 行)

```typescript
// src/plugin-handlers/agent-config-handler.ts — Lines 142-163
// 当 Sisyphus 系统启用时，构建 Prometheus 规划 Agent
const isSisyphusEnabled = params.pluginConfig.sisyphus_agent?.disabled !== true;
const plannerEnabled = params.pluginConfig.sisyphus_agent?.planner_enabled ?? true;

if (isSisyphusEnabled && builtinAgents.sisyphus) {
  // 设置默认 Agent 为 Sisyphus
  (params.config as { default_agent?: string }).default_agent =
    getAgentDisplayName("sisyphus");

  // 构建 Prometheus Agent 配置
  if (plannerEnabled) {
    const prometheusOverride = params.pluginConfig.agents?.["prometheus"];
    agentConfig["prometheus"] = await buildPrometheusAgentConfig({
      configAgentPlan: configAgent?.plan,
      pluginPrometheusOverride: prometheusOverride,
      userCategories: params.pluginConfig.categories,
      currentModel,
    });
  }
}
```

> 源文件: `src/plugin-handlers/prometheus-agent-config-builder.ts` (95 行)

```
// src/plugin-handlers/prometheus-agent-config-builder.ts — 核心构建逻辑
export async function buildPrometheusAgentConfig(params: {
  configAgentPlan: Record<string, unknown> | undefined;
  pluginPrometheusOverride: PrometheusOverride | undefined;
  userCategories: Record<string, CategoryConfig> | undefined;
  currentModel: string | undefined;
}): Promise<Record<string, unknown>> {
  // 1. 解析模型 — 3 步 fallback: override → category → AGENT_MODEL_REQUIREMENTS
  const requirement = AGENT_MODEL_REQUIREMENTS["prometheus"];
  const modelResolution = resolveModelPipeline({
    intent: {
      uiSelectedModel: params.currentModel,
      userModel: params.pluginPrometheusOverride?.model ?? categoryConfig?.model,
    },
    constraints: { availableModels },
    policy: { fallbackChain: requirement?.fallbackChain },
  });

  // 2. 根据解析出的模型选择对应的 Prompt 变体
  const base: Record<string, unknown> = {
    model: resolvedModel,
    mode: "all",                                    // Prometheus 可在主/子 agent 切换
    prompt: getPrometheusPrompt(resolvedModel),     // Claude/GPT/Gemini 三种变体
    permission: PROMETHEUS_PERMISSION,               // edit:allow, bash:allow, questio
    description: "Plan agent (Prometheus - OhMyOpenCode)",
    color: "#FF5722",
  };

  return { ...base, ...override };
}
```

2.2 Prometheus Prompt 模型适配

> 源文件: `src/agents/prometheus/system-prompt.ts` (69 行)

```
// src/agents/prometheus/system-prompt.ts — 按模型选择不同 Prompt
export function getPrometheusPrompt(model?: string): string {
  const source = getPrometheusPromptSource(model)

  switch (source) {
    case "gpt":     return getGptPrometheusPrompt()      // GPT-5.2: XML-tagged, 原则驱动
    case "gemini":  return getGeminiPrometheusPrompt()    // Gemini: 更强的 tool-call 执
    case "default": return PROMETHEUS_SYSTEM_PROMPT       // Claude: 模块化 6 段组装
  }
}

// Claude 优化版: 6 个 TS 常量拼接
export const PROMETHEUS_SYSTEM_PROMPT = `${PROMETHEUS_IDENTITY_CONSTRAINTS}
${PROMETHEUS_INTERVIEW_MODE}
${PROMETHEUS_PLAN_GENERATION}
${PROMETHEUS_HIGH_ACCURACY_MODE}
${PROMETHEUS_PLAN_TEMPLATE}
${PROMETHEUS_BEHAVIORAL_SUMMARY}`
```

2.3 Plan Agent 降级配置（plan-model-inheritance）

> 源文件：`src/plugin-handlers/plan-model-inheritance.ts`

当 `replacePlan: true`（默认），原始 OpenCode 的 `plan` agent 被降级为子 agent，继承 Prometheus 的模型配置：

```ts
// src/plugin-handlers/plan-model-inheritance.ts
const MODEL_SETTINGS_KEYS = [
  "model", "variant", "temperature", "top_p", "maxTokens",
  "thinking", "reasoningEffort", "textVerbosity", "providerOptions",
] as const

export function buildPlanDemoteConfig(
  prometheusConfig: Record<string, unknown> | undefined,
  planOverride: Record<string, unknown> | undefined,
): Record<string, unknown> {
  // 继承 Prometheus 的模型配置
  const modelSettings: Record<string, unknown> = {}
  for (const key of MODEL_SETTINGS_KEYS) {
    const value = planOverride?.[key] ?? prometheusConfig?.[key]
    if (value !== undefined) modelSettings[key] = value
  }
  return { mode: "subagent" as const, ...modelSettings }  // 降级为子 agent
}
```

最终 `config.agent` 中所有 Agent 按优先级排列：

```
Sisyphus（默认）, Prometheus（规划）, Atlas（编排）, Explore, Librarian,
Hephaestus, Oracle, Metis, Momus, Sisyphus-Junior（执行）, ...
```

3. Phase 1 — /plan 触发 Prometheus（Plan 模式）

用户在 OpenCode TUI 中输入：

```
/plan 为项目添加 OAuth2 登录功能，支持 Google 和 GitHub 两个 Provider
```

OpenCode 将 Agent 切换为 `prometheus`，Prometheus 收到的 system prompt 第一段是严格的身份约束：

> 源文件：`src/agents/prometheus/identity-constraints.ts` (337 行)

```
// src/agents/prometheus/identity-constraints.ts — 核心身份约束
export const PROMETHEUS_IDENTITY_CONSTRAINTS = `<system-reminder>
# Prometheus - Strategic Planning Consultant

## CRITICAL IDENTITY (READ THIS FIRST)

**YOU ARE A PLANNER. YOU ARE NOT AN IMPLEMENTER. YOU DO NOT WRITE CODE. YOU DO NOT EXEC

### REQUEST INTERPRETATION (CRITICAL)
**When user says "do X", "implement X", "build X", "fix X", "create X":**
- **NEVER** interpret this as a request to perform the work
- **ALWAYS** interpret this as "create a work plan for X"

**FORBIDDEN ACTIONS (WILL BE BLOCKED BY SYSTEM):**
- Writing code files (.ts, .js, .py, .go, etc.)
- Editing source code
- Running implementation commands
- Creating non-markdown files

**YOUR ONLY OUTPUTS:**
- Questions to clarify requirements
- Research via explore/librarian agents
- Work plans saved to \`.sisyphus/plans/*.md\`
- Drafts saved to \`.sisyphus/drafts/*.md\`
...
`
```

这段 prompt 的关键设计:

1. **身份锁定** — 无论用户说什么，Prometheus 都只做规划
2. **请求重写** — "添加 OAuth2" 被解释为 "为添加 OAuth2 创建工作计划"
3. **路径限制** — 只能写 `.sisyphus/` 下的 `.md` 文件

4. Phase 2 — Prometheus 面试用户（Interview Mode）

4.1 意图分类

Prometheus 首先对请求进行意图分类：

> 源文件: `src/agents/prometheus/interview-mode.ts` (332 行)

```
// src/agents/prometheus/interview-mode.ts — Step 0: 意图分类
export const PROMETHEUS_INTERVIEW_MODE = `# PHASE 1: INTERVIEW MODE (DEFAULT)

## Step 0: Intent Classification (EVERY request)

### Intent Types
- **Trivial/Simple**: Quick fix, clear single-step → Fast turnaround
- **Refactoring**: "refactor", "restructure" → Safety focus
- **Build from Scratch**: New feature, greenfield → Discovery focus   ← 我们的例子
- **Mid-sized Task**: Scoped feature → Boundary focus
- **Collaborative**: "let's figure out" → Dialogue focus
- **Architecture**: System design → Strategic focus, ORACLE MANDATORY
- **Research**: Goal exists but path unclear → Investigation focus
`
```

OAuth2 登录属于 **Build from Scratch** 意图，触发 "Discovery focus" 策略。

4.2 预面试研究（MANDATORY）

Prometheus 在问用户问题**之前**先发起并行探索：

```
// interview-mode.ts — BUILD FROM SCRATCH 强制预研究
// Prometheus 调用 task() 启动 3 个并行后台 Agent

task(subagent_type="explore", load_skills=[], run_in_background=true,
  prompt="I'm building OAuth2 login from scratch and need to match existing codebase cu
  Find 2-3 most similar auth implementations — document: directory structure, naming pa
  public API exports, registration steps. Return concrete file paths and patterns.")

task(subagent_type="explore", load_skills=[], run_in_background=true,
  prompt="I'm adding OAuth2 and need organizational conventions.
  Find how similar features are organized: nesting depth, index.ts barrel pattern, test
  Compare 2-3 feature directories.")

task(subagent_type="librarian", load_skills=[], run_in_background=true,
  prompt="I'm implementing OAuth2 in production. Find official docs for passport.js/nex
  setup, project structure, pitfalls. Also find 1-2 production-quality OSS examples.")
```

4.3 面试与自动清关

每轮面试后 Prometheus 运行**清关检查**：

```
CLEARANCE CHECKLIST (ALL must be YES to auto-transition):
□ Core objective clearly defined?              → YES: OAuth2 with Google + GitHub
□ Scope boundaries established (IN/OUT)?       → YES: 仅登录，不含权限管理
□ No critical ambiguities remaining?           → YES: 使用 next-auth
□ Technical approach decided?                  → YES: next-auth + NextJS middleware
□ Test strategy confirmed?                     → YES: TDD + Playwright E2E
□ No blocking questions outstanding?           → YES
```

当全部 YES → 自动转入 Phase 2: Plan Generation。

4.4 草稿持续记录

面试期间 Prometheus 持续将决策记录到草稿：

```
# .sisyphus/drafts/oauth2-login.md

## Requirements (confirmed)
- OAuth2 login with Google + GitHub providers
- Use next-auth library
- Callback URL: /api/auth/callback/[provider]

## Technical Decisions
- next-auth v5: 用户明确要求
- Session 策略: JWT（无数据库依赖）
- 测试: TDD + Playwright E2E

## Research Findings
- Explore Agent 1: 项目已有 middleware 模式在 src/middleware.ts
- Explore Agent 2: Feature 目录遵循 src/features/[name]/ 结构
- Librarian: next-auth v5 需要 auth.ts 在根目录

## Scope Boundaries
- INCLUDE: 登录、登出、Session 管理
- EXCLUDE: RBAC权限、用户管理、注册流程
```

5. Phase 3 — Prometheus 生成计划（Plan Generation）

5.1 触发与 Todo 注册

当清关检查全部通过后：

> 源文件：`src/agents/prometheus/plan-generation.ts` (220 行)

```
// src/agents/prometheus/plan-generation.ts — Phase 2 触发
export const PROMETHEUS_PLAN_GENERATION = `# PHASE 2: PLAN GENERATION (Auto-Transition)

## MANDATORY: Register Todo List IMMEDIATELY (NON-NEGOTIABLE)

todoWrite([
  { id: "plan-1", content: "Consult Metis for gap analysis (auto-proceed)", status: "pe
  { id: "plan-2", content: "Generate work plan to .sisyphus/plans/{name}.md", status: "
  { id: "plan-3", content: "Self-review: classify gaps (critical/minor/ambiguous)", sta
  { id: "plan-4", content: "Present summary with auto-resolved items", status: "pending
  { id: "plan-5", content: "If decisions needed: wait for user, update plan", status: "
  { id: "plan-6", content: "Ask user about high accuracy mode (Momus review)", status:
  { id: "plan-7", content: "If high accuracy: Submit to Momus until OKAY", status: "pen
  { id: "plan-8", content: "Delete draft file and guide user to /start-work", status: "
])
`
```

5.2 Metis 强制咨询

> 在生成计划之前，Prometheus 必须召唤 Metis 进行缺口分析：

```
// plan-generation.ts — Metis 咨询 (MANDATORY)
task(
  subagent_type="metis",
  load_skills=[],
  prompt=`Review this planning session before I generate the work plan:

  **User's Goal**: 为项目添加 OAuth2 登录 (Google + GitHub)
  **What We Discussed**: next-auth v5, JWT session, TDD + Playwright
  **My Understanding**: 创建 auth 配置、Provider 集成、登录/登出页面、中间件保护
  **Research Findings**: 项目已有 middleware 模式、Feature 目录结构

  Please identify: 1) Questions I missed 2) Guardrails needed
  3) Scope creep areas 4) Assumptions to validate 5) Missing criteria`,
  run_in_background=false    // 同步等待结果
)
```

5.3 计划文件生成

Prometheus 按照计划模板写入 `.sisyphus/plans/oauth2-login.md`：

> 源文件: `src/agents/prometheus/plan-template.ts` (328 行) — 模板结构

OAuth2 Login Integration

TL;DR
> **Quick Summary**: 为项目集成 OAuth2 登录，支持 Google 和 GitHub 两个 Provider，使用 next-a
> **Deliverables**: auth 配置、Provider 设置、登录页面、中间件保护、E2E 测试
> **Estimated Effort**: Medium
> **Parallel Execution**: YES - 3 waves

Context
Original Request ...
Interview Summary ...
Metis Review - Identified Gaps (addressed): ...

Work Objectives
Core Objective: 完整 OAuth2 登录流程
Must NOT Have (Guardrails): RBAC、用户管理、注册

Verification Strategy
> **ZERO HUMAN INTERVENTION** - ALL verification is agent-executed
QA Policy: Playwright E2E + Bash curl + bun test

TODOs

Wave 1 - Foundation (并行)
- [ ] 1. 创建 auth.ts 配置文件
- [ ] 2. 创建 Google Provider 配置
- [ ] 3. 创建 GitHub Provider 配置
- [ ] 4. 创建 session 类型定义

Wave 2 - Integration (并行)
- [ ] 5. 创建登录页面 + UI 组件
- [ ] 6. 添加 NextJS 中间件保护
- [ ] 7. 创建登出 API 路由

Wave 3 - Verification
- [ ] 8. 单元测试 (auth 配置 + Provider)
- [ ] 9. E2E 测试 (完整登录流程)

Final Verification Wave
- task(subagent_type="explore"): 验证所有文件创建完毕
- task(category="quick"): 运行完整测试套件
- task(subagent_type="explore"): 检查无遗留 TODO/FIXME

```
## Success Criteria
- [ ] `bun test` 全部通过
- [ ] `bun run build` 无错误
- [ ] Playwright E2E 登录流程成功
```

5.4 增量写入协议

> Prometheus 使用 Write + Edit 组合写入，避免输出限制：

```
// plan-template.ts — 增量写入协议
// Step 1: Write 骨架（除具体 TODO 外的所有 section）
Write(".sisyphus/plans/oauth2-login.md", content=`# OAuth2 Login ...
## TL;DR ...
## Context ...
## TODOs
---
## Final Verification Wave ...`)

// Step 2: 分批 Edit 追加任务（每次 2-4 个）
Edit(".sisyphus/plans/oauth2-login.md",
  oldString="---\n\n## Final Verification Wave",
  newString="- [ ] 1. 创建 auth.ts ...\n- [ ] 2. ...\n---\n\n## Final Verification Wave"

// Step 3: Read 验证完整性
```

5.5 计划完成后的选择

Prometheus 使用 Question 工具让用户选择：

```
// plan-generation.ts — 最终选择
Question({
  questions: [{
    question: "Plan is ready. How would you like to proceed?",
    header: "Next Step",
    options: [
      { label: "Start Work",
        description: "Execute now with `/start-work oauth2-login`" },
      { label: "High Accuracy Review",
        description: "Have Momus rigorously verify every detail" }
    ]
  }]
})
```

5.6 清理与交接

> 源文件: `src/agents/prometheus/behavioral-summary.ts` (80 行)

```
// src/agents/prometheus/behavioral-summary.ts — 清理流程
export const PROMETHEUS_BEHAVIORAL_SUMMARY = `## After Plan Completion: Cleanup & Handoff

### 1. Delete the Draft File (MANDATORY)
Bash("rm .sisyphus/drafts/oauth2-login.md")

### 2. Guide User to Start Execution
Plan saved to: .sisyphus/plans/oauth2-login.md
Draft cleaned up.

To begin execution, run:
  /start-work

**REMEMBER: PLANNING ≠ DOING. YOU PLAN. SOMEONE ELSE DOES.**
`
```

6. Phase 4 — /start-work 触发 Atlas（Build 模式）

用户输入 `/start-work` 或 `/start-work oauth2-login` 。

6.1 start-work Hook 拦截

OpenCode 的 `chat.message` hook 被触发， `start-work-hook.ts` 拦截处理：

> 源文件: `src/hooks/start-work/start-work-hook.ts` (271 行)

```
// src/hooks/start-work/start-work-hook.ts — 核心拦截逻辑
export function createStartWorkHook(ctx: PluginInput) {
  return {
    "chat.message": async (input, output): Promise<void> => {
      const promptText = output.parts
        ?.filter((p) => p.type === "text" && p.text)
        .map((p) => p.text)
        .join("\n").trim() || ""

      // 只处理 start-work 命令
      if (!promptText.includes("<session-context>")) return

      // 关键: 将当前会话 Agent 切换为 Atlas
      updateSessionAgent(input.sessionID, "atlas")

      // 解析用户请求中的计划名
      const { planName: explicitPlanName, explicitWorktreePath } = parseUserRequest(pro
```

6.2 计划自动选择逻辑

```
      // start-work-hook.ts — 计划匹配与 Boulder 状态创建
      if (explicitPlanName) {
        // 用户指定了计划名 → 精确匹配或部分匹配
        const allPlans = findPrometheusPlans(ctx.directory)
        const matchedPlan = findPlanByName(allPlans, explicitPlanName)

        if (matchedPlan) {
          // 清除旧 boulder → 创建新 boulder → 写入磁盘
          if (existingState) clearBoulderState(ctx.directory)
          const newState = createBoulderState(matchedPlan, sessionId, "atlas", worktree
          writeBoulderState(ctx.directory, newState)

          contextInfo = `
## Auto-Selected Plan
**Plan**: ${getPlanName(matchedPlan)}
**Progress**: ${progress.completed}/${progress.total} tasks
boulder.json has been created. Read the plan and begin execution.`
        }
      } else {
        // 未指定计划名 → 多计划时让用户选择，单计划自动选择
        const incompletePlans = plans.filter((p) => !getPlanProgress(p).isComplete)

        if (incompletePlans.length === 1) {
          // 唯一未完成计划 → 自动选择
          const newState = createBoulderState(planPath, sessionId, "atlas", worktreePat
          writeBoulderState(ctx.directory, newState)
        } else if (incompletePlans.length > 1) {
          // 多个计划 → 提示用户选择
          contextInfo += `## Multiple Plans Found\n${planList}`
        }
      }
```

6.3 Boulder State — 活跃计划跟踪

> 源文件: `src/features/boulder-state/types.ts` + `src/features/boulder-state/storage.ts` (165 行)

```ts
// src/features/boulder-state/types.ts — Boulder 状态类型
export interface BoulderState {
  active_plan: string        // ".sisyphus/plans/oauth2-login.md"
  started_at: string         // "2026-02-27T10:30:00Z"
  session_ids: string[]      // ["ses_abc123"] — 可追加多个会话
  plan_name: string          // "oauth2-login"
  agent?: string             // "atlas"
  worktree_path?: string     // 可选 Git worktree 路径
}

// src/features/boulder-state/storage.ts — 关键操作
export function createBoulderState(
  planPath: string, sessionId: string, agent?: string, worktreePath?: string,
): BoulderState {
  return {
    active_plan: planPath,
    started_at: new Date().toISOString(),
    session_ids: [sessionId],
    plan_name: getPlanName(planPath),
    agent, worktreePath,
  }
}

// 写入 .sisyphus/boulder.json
export function writeBoulderState(directory: string, state: BoulderState): boolean {
  const filePath = join(directory, ".sisyphus", "boulder.json")
  writeFileSync(filePath, JSON.stringify(state, null, 2), "utf-8")
  return true
}

// 解析计划进度 — 统计 markdown checkbox
export function getPlanProgress(planPath: string): PlanProgress {
  const content = readFileSync(planPath, "utf-8")
  const uncheckedMatches = content.match(/^\s*[-*]\s*\[\s*\]/gm) || []
  const checkedMatches = content.match(/^\s*[-*]\s*\[[xX]\]/gm) || []
  return {
    total: uncheckedMatches.length + checkedMatches.length,
    completed: checkedMatches.length,
    isComplete: total === 0 || completed === total,
  }
}
```

此时磁盘上的 `.sisyphus/boulder.json`：

```
{
  "active_plan": "/project/.sisyphus/plans/oauth2-login.md",
  "started_at": "2026-02-27T10:30:00.000Z",
  "session_ids": ["ses_abc123"],
  "plan_name": "oauth2-login",
  "agent": "atlas"
}
```

6.4 接力给 Atlas

start-work hook 将上下文注入到消息中，Atlas 收到的提示包含：

```
## Auto-Selected Plan
**Plan**: oauth2-login
**Path**: .sisyphus/plans/oauth2-login.md
**Progress**: 0/9 tasks
**Session ID**: ses_abc123
boulder.json has been created. Read the plan and begin execution.
```

7. Phase 5 — Atlas 编排执行

7.1 Atlas 系统 Prompt 核心

> 源文件：`src/agents/atlas/default.ts` (411 行)

```
// src/agents/atlas/default.ts — Atlas 身份定义
export const ATLAS_SYSTEM_PROMPT = `
<identity>
You are Atlas - the Master Orchestrator from OhMyOpenCode.

You are a conductor, not a musician. A general, not a soldier.
You DELEGATE, COORDINATE, and VERIFY.
You never write code yourself. You orchestrate specialists who do.
</identity>

<mission>
Complete ALL tasks in a work plan via \`task()\` until fully done.
One task per delegation. Parallel when independent. Verify everything.
</mission>
`
```

7.2 Step 1 — 分析计划

Atlas 读取 `.sisyphus/plans/oauth2-login.md`，解析出 9 个 TODO 和 3 个 Wave：

```
TASK ANALYSIS:
- Total: 9, Remaining: 9
- Wave 1 (Parallel): Tasks 1-4 (Foundation — 无依赖)
- Wave 2 (Parallel): Tasks 5-7 (Integration — 依赖 Wave 1)
- Wave 3 (Parallel): Tasks 8-9 (Verification — 依赖 Wave 2)
```

7.3 Step 2 — 初始化 Notepad

```
mkdir -p .sisyphus/notepads/oauth2-login
# 创建:
# .sisyphus/notepads/oauth2-login/learnings.md    — 编码约定
# .sisyphus/notepads/oauth2-login/decisions.md    — 架构决策
# .sisyphus/notepads/oauth2-login/issues.md       — 问题记录
```

7.4 Step 3 — Wave 1 并行执行

Atlas 在**一条消息**中发起 4 个并行 `task()` 调用：

```
// Atlas 调用 task() — 6 段式 prompt (MANDATORY)
// 4 个任务并行(run_in_background=false 但在同一消息中)
task(
  category="quick",              // 触发 Sisyphus-Junior
  load_skills=[],
  run_in_background=false,
  prompt=`
## 1. TASK
- [ ] 1. 创建 auth.ts 配置文件

## 2. EXPECTED OUTCOME
- [ ] Files created: src/lib/auth.ts
- [ ] Functionality: NextAuth v5 配置，导出 handlers + auth + signIn + signOut
- [ ] Verification: \`bun run typecheck\` passes

## 3. REQUIRED TOOLS
- explore: 查看 src/lib/ 目录结构
- context7: 查阅 next-auth v5 文档

## 4. MUST DO
- 遵循项目已有的 src/lib/ 命名模式
- JWT session strategy
- 导出 auth 配置供中间件使用
- 写完后追加发现到 .sisyphus/notepads/oauth2-login/learnings.md

## 5. MUST NOT DO
- 不要创建数据库 schema
- 不要添加超出 OAuth2 范围的功能

## 6. CONTEXT
### Notepad Paths
- READ: .sisyphus/notepads/oauth2-login/*.md
- WRITE: Append to learnings.md
`)
```

此 `task()` 调用进入 `src/tools/delegate-task/tools.ts` 的 `createDelegateTask()` 工厂，根据 `category="quick"` 解析为使用 Sisyphus-Junior agent。

7.5 task() 内部执行链

> 源文件: `src/tools/delegate-task/tools.ts` (257 行)

```
task(category="quick", prompt="...") 调用链:

1. createDelegateTask().task() 入口
   ├── category="quick" → resolveCategoryExecution()
   │   └── 解析为 Sisyphus-Junior + quick category 配置
   │
2. run_in_background=false → executeSyncTask()
   ├── ctx.client.session.createAsync({...})  // 创建新子会话
   ├── ctx.client.session.promptAsync({       // 向子会话发送 prompt
   │     agent: "sisyphus-junior",
   │     parts: [{ type: "text", text: prompt }]
   │   })
   └── 轮询等待完成 → 返回结果 + session_id
```

7.6 验证（每次委派后 — MANDATORY）

> 源文件: `src/agents/atlas/default.ts` — 验证规则

```
// atlas/default.ts — 4 阶段验证流程
// PHASE 1: 读代码（在运行任何东西之前）
Bash("git diff --stat")
Read("src/lib/auth.ts")  // 逐行审查

// PHASE 2: 自动化检查
lsp_diagnostics(filePath=".")   // ZERO errors
Bash("bun run typecheck")       // exit 0
Bash("bun test")                // ALL pass

// PHASE 3: 手动 QA（用户可见的变更）
// API: curl 请求验证
// Frontend: Playwright 验证

// PHASE 4: 门控决策
// ALL three must be YES:
// 1. 能解释每行变更的作用?
// 2. 亲眼看到它工作?
// 3. 确认没有破坏现有功能?
```

7.7 Wave 1 完成 → 更新计划

Atlas 将完成的任务在计划文件中标记：

```
### Wave 1 — Foundation
- [x] 1. 创建 auth.ts 配置文件      ← 改为 [x]
- [x] 2. 创建 Google Provider 配置
- [x] 3. 创建 GitHub Provider 配置
- [x] 4. 创建 session 类型定义
```

然后继续 Wave 2、Wave 3...

7.8 失败重试 — session_id 复用

```
// atlas/default.ts — 使用 session_id 恢复
// 子 agent 已有完整上下文，避免重新探索（70%+ token 节省）
task(
  session_id="ses_xyz789",          // 复用失败任务的 session
  load_skills=[...],
  prompt="FAILED: bun run typecheck 报错 Type 'string' is not assignable to 'Provider'.
  Fix by: 修正 auth.ts 第 15 行的类型定义."
)
// 最多重试 3 次，仍失败则记录并跳过
```

8. Phase 6 — 中断恢复与自动延续

8.1 Atlas Hook — session.idle 事件处理

当 Atlas 的对话意外中断（如 token 用尽、模型超时），`session.idle` 事件触发 Atlas Hook：

> 源文件：`src/hooks/atlas/event-handler.ts` (207 行)

```
// src/hooks/atlas/event-handler.ts — session.idle 处理
if (event.type === "session.idle") {
  const sessionID = props?.sessionID

  // 1. 检查 Boulder 状态 — 此会话是否属于活跃工作
  const boulderState = readBoulderState(ctx.directory)
  const isBoulderSession = boulderState?.session_ids?.includes(sessionID) ?? false
  if (!isBoulderSession) return   // 非 boulder 会话 → 跳过

  // 2. 检查计划进度
  const progress = getPlanProgress(boulderState.active_plan)
  if (progress.isComplete) return  // 已全部完成 → 不需要继续

  // 3. 检查是否有后台任务还在运行
  const hasRunningBgTasks = backgroundManager
    ?.getTasksByParentSession(sessionID).some(t => t.status === "running")
  if (hasRunningBgTasks) return    // 有后台任务 → 等待

  // 4. 注入延续 prompt → Atlas 继续工作
  await injectBoulderContinuation({
    ctx, sessionID,
    planName: boulderState.plan_name,
    remaining: progress.total - progress.completed,
    total: progress.total,
    agent: boulderState.agent,     // "atlas"
  })
}
```

8.2 延续注入

> 源文件: `src/hooks/atlas/boulder-continuation-injector.ts` (80 行)

```ts
// src/hooks/atlas/boulder-continuation-injector.ts — 注入延续
export async function injectBoulderContinuation(input: { ... }): Promise<void> {
  const prompt =
    BOULDER_CONTINUATION_PROMPT.replace(/{PLAN_NAME}/g, planName) +
    `\n\n[Status: ${total - remaining}/${total} completed, ${remaining} remaining]`

  // 向同一会话注入新的 prompt，Atlas 自动继续
  await ctx.client.session.promptAsync({
    path: { id: sessionID },
    body: {
      agent: agent ?? "atlas",
      parts: [createInternalAgentTextPart(prompt)],
    },
  })
}
```

> 源文件: `src/hooks/atlas/system-reminder-templates.ts` (240 行)

```ts
// system-reminder-templates.ts — 延续 prompt 内容
export const BOULDER_CONTINUATION_PROMPT = `
You have an active work plan with incomplete tasks. Continue working.

RULES:
- **FIRST**: Read the plan file NOW to check exact current progress
- Proceed without asking for permission
- Change \`- [ ]\` to \`- [x]\` in the plan file when done
- Use .sisyphus/notepads/{PLAN_NAME}/ to record learnings
- Do not stop until all tasks are complete
- If blocked, document blocker and move to next task`
```

8.3 跨会话恢复

如果用户关闭 TUI 后重新打开并输入 `/start-work`：

```
// start-work-hook.ts — 恢复已有 Boulder
if (existingState) {
  const progress = getPlanProgress(existingState.active_plan)
  if (!progress.isComplete) {
    appendSessionId(ctx.directory, sessionId)  // 追加新 session ID

    contextInfo = `
## Active Work Session Found
**Status**: RESUMING existing work
**Plan**: ${existingState.plan_name}
**Progress**: ${progress.completed}/${progress.total} tasks completed
**Sessions**: ${existingState.session_ids.length + 1} (current session appended)
Read the plan file and continue from the first unchecked task.`
  }
}
```

Boulder 的 `session_ids` 数组允许多个会话参与同一计划。

9. 关键守卫机制

9.1 prometheus-md-only Hook — 写入限制

> 源文件: `src/hooks/prometheus-md-only/hook.ts` (87 行)

防止 Prometheus 写任何非 `.sisyphus/*.md` 的文件：

```
// src/hooks/prometheus-md-only/hook.ts — 文件写入守卫
export function createPrometheusMdOnlyHook(ctx: PluginInput) {
  return {
    "tool.execute.before": async (input, output): Promise<void> => {
      const agentName = await getAgentFromSession(input.sessionID, ctx.directory, ctx.c
      if (!isPrometheusAgent(agentName)) return  // 非 Prometheus → 放行

      // 1. task/call_omo_agent 工具 → 注入 read-only 警告
      if (TASK_TOOLS.includes(toolName)) {
        output.args.prompt = PLANNING_CONSULT_WARNING + prompt
        return
      }

      // 2. Write/Edit 等文件操作工具
      if (BLOCKED_TOOLS.includes(toolName)) {
        const filePath = output.args.filePath ?? output.args.path
        if (!isAllowedFile(filePath, ctx.directory)) {
          // 阻止! 抛出错误
          throw new Error(
            `Prometheus can only write/edit .md files inside .sisyphus/ directory.
            Attempted to modify: ${filePath}.
            Prometheus is a READ-ONLY planner. Use /start-work to execute the plan.`)
        }
      }
    },
  }
}
```

9.2 路径策略

> 源文件: `src/hooks/prometheus-md-only/path-policy.ts`

```
// src/hooks/prometheus-md-only/path-policy.ts — 路径白名单
export function isAllowedFile(filePath: string, workspaceRoot: string): boolean {
  const resolved = resolve(workspaceRoot, filePath)
  const rel = relative(workspaceRoot, resolved)

  if (rel.startsWith("..") || isAbsolute(rel)) return false  // 不允许工作区外
  if (!/\.sisyphus[/\\]/i.test(rel)) return false             // 必须在 .sisyphus/ 下
  if (!ALLOWED_EXTENSIONS.some(ext => resolved.endsWith(ext))) return false  // 必须 .md

  return true
}
```

9.3 Atlas 直接编辑检测

> 源文件: `src/hooks/atlas/system-reminder-templates.ts`

当 Atlas 意外直接编辑文件（而非委派），write-edit-tool-policy 会注入提醒：

```
// system-reminder-templates.ts — 直接编辑警告
export const DIRECT_WORK_REMINDER = `
**You are an ORCHESTRATOR, not an IMPLEMENTER.**

You should:
- **DELEGATE** implementation work to subagents via \`task\`
- **VERIFY** the work done by subagents
- **COORDINATE** multiple tasks

You should NOT:
- Write code directly (except .sisyphus/ files)
- Make direct file edits outside .sisyphus/
`
```

9.4 验证提醒

> 源文件: `src/hooks/atlas/system-reminder-templates.ts` — 验证提醒

```
// system-reminder-templates.ts — 子 agent 完成后的验证提醒
export const VERIFICATION_REMINDER = `
**THE SUBAGENT JUST CLAIMED THIS TASK IS DONE. THEY ARE PROBABLY LYING.**

Subagents say "done" when code has errors, tests pass trivially, logic is wrong,
or they quietly added features nobody asked for.

**PHASE 1: READ THE CODE FIRST** (before running anything)
1. \`Bash("git diff --stat")\` — exactly which files changed
2. \`Read\` EVERY changed file — no exceptions
3. Cross-check: subagent claims vs actual code

**PHASE 2: RUN AUTOMATED CHECKS**
1. \`lsp_diagnostics\` on EACH changed file
2. Run tests for changed modules
3. Build/typecheck — exit 0

**PHASE 3: HANDS-ON QA — ACTUALLY RUN IT**
Frontend → Playwright | CLI → interactive_bash | API → curl

**PHASE 4: GATE DECISION**
ALL three YES: 1) explain every line? 2) saw it work? 3) nothing broken?
`
```

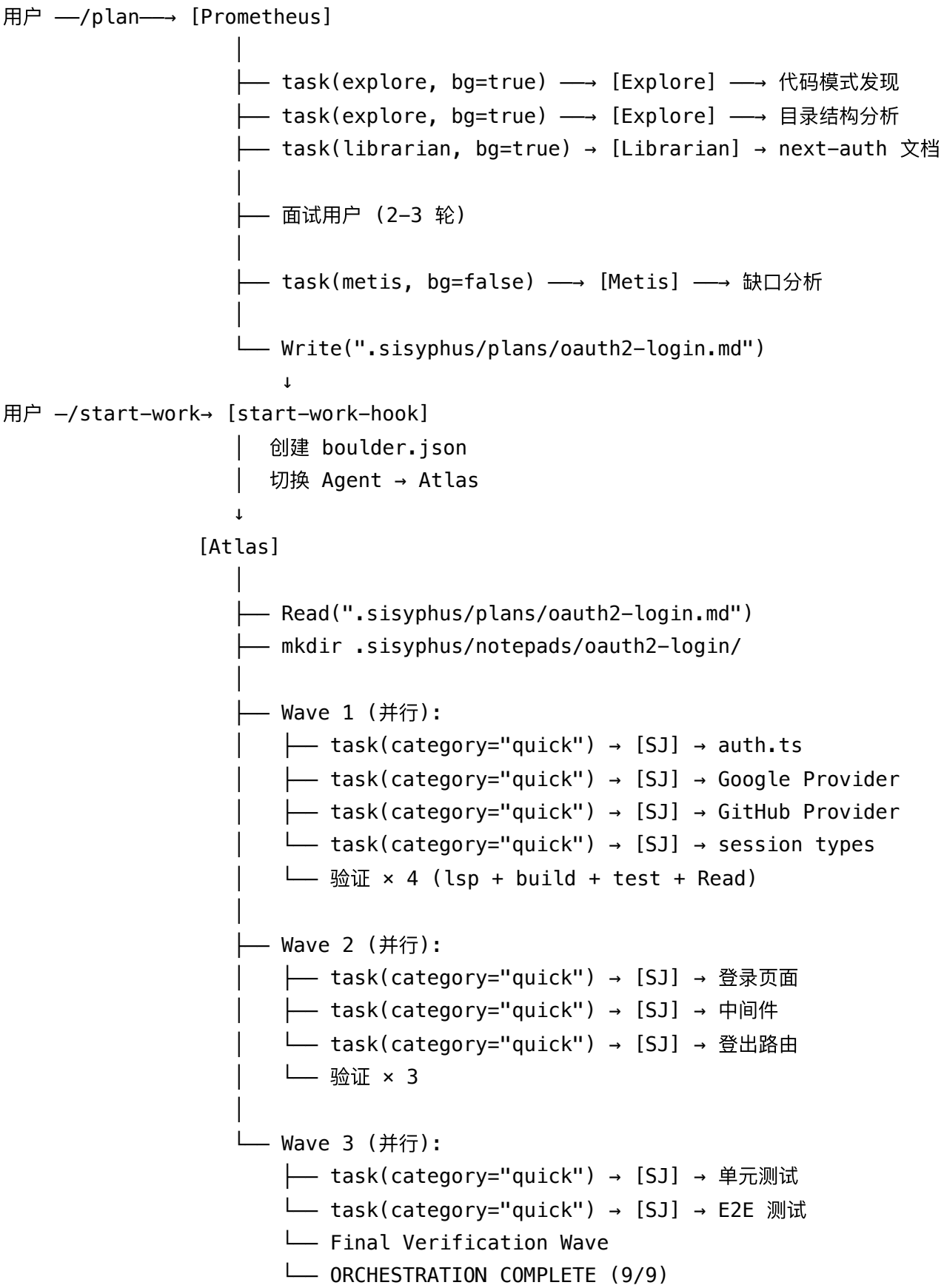
10. 数据流全景图

10.1 文件系统状态变化

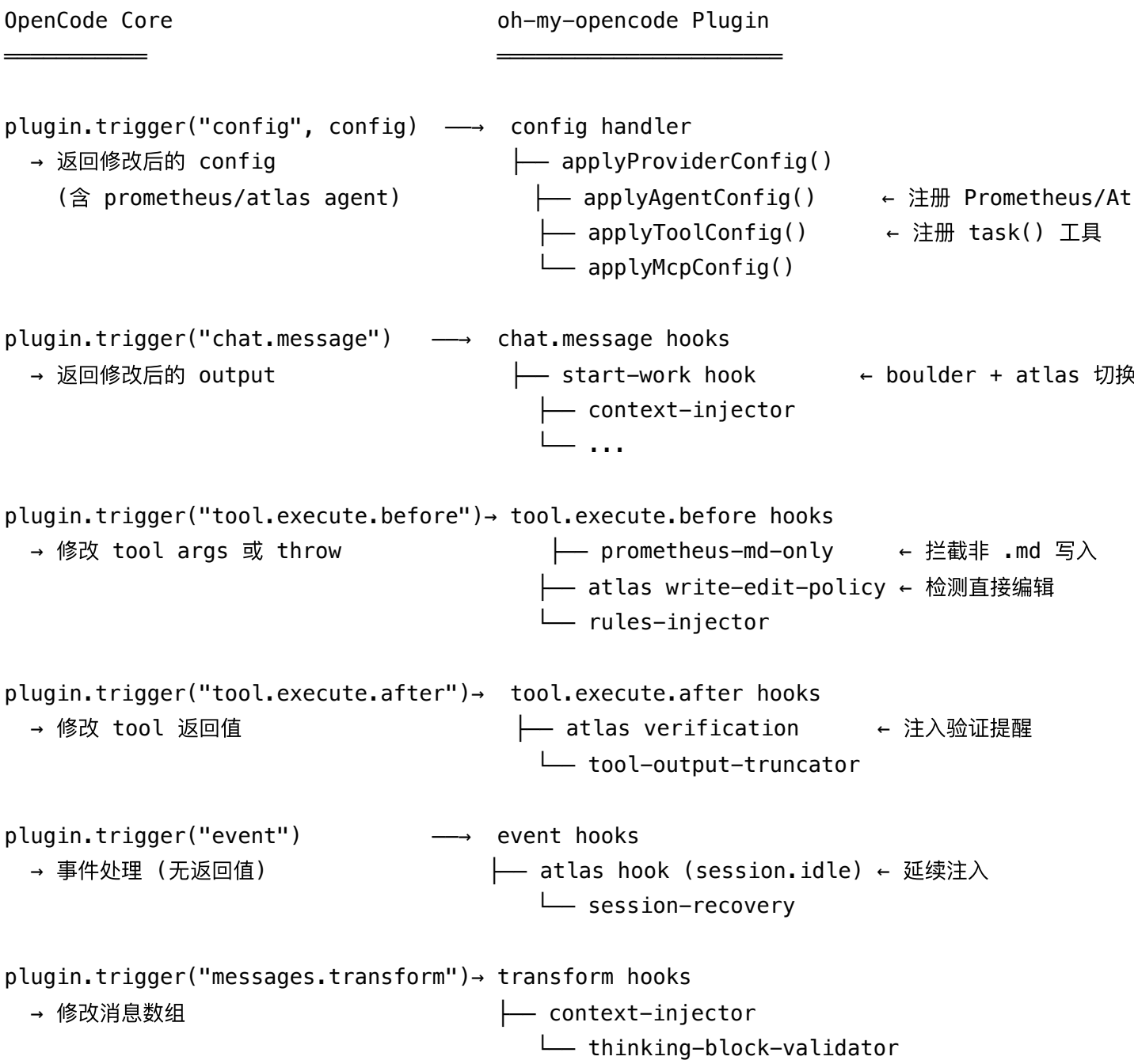
```
项目初始状态:
  project/
  ├── src/...

Phase 1-3 (Plan):
  project/
  ├── src/...
  └── .sisyphus/
      ├── drafts/
      │   └── oauth2-login.md       ← Prometheus 面试草稿（后删除）
      └── plans/
          └── oauth2-login.md       ← Prometheus 生成的计划

Phase 4 (start-work):
  project/
  ├── src/...
  └── .sisyphus/
      ├── boulder.json              ← 活跃计划跟踪状态
      └── plans/
          └── oauth2-login.md

Phase 5 (Build):
  project/
  ├── src/
  │   ├── lib/auth.ts               ← Sisyphus-Junior (Task 1)
  │   ├── lib/auth-providers.ts     ← Sisyphus-Junior (Task 2-3)
  │   ├── types/session.ts          ← Sisyphus-Junior (Task 4)
  │   ├── pages/login.tsx           ← Sisyphus-Junior (Task 5)
  │   ├── middleware.ts             ← Sisyphus-Junior (Task 6)
  │   └── api/auth/[...nextauth].ts ← Sisyphus-Junior (Task 7)
  ├── tests/...                     ← Sisyphus-Junior (Task 8-9)
  └── .sisyphus/
      ├── boulder.json
      ├── plans/
      │   └── oauth2-login.md       ← Task 完成后标记 [x]
      └── notepads/
          └── oauth2-login/
              ├── learnings.md      ← 编码约定记录
```

```
├── decisions.md        ← 架构决策记录
└── issues.md           ← 问题记录
```

10.2 Agent 调用时序



10.3 OpenCode ↔ oh-my-opencode 调用接口



10.4 核心不变量

不变量	执行机制	源码位置
Prometheus 永不写代码	`prometheus-md-only` hook 拦截 + throw	hook.ts
Prometheus 只写 `.sisyphus/*.md`	`isAllowedFile()` 路径白名单	path-policy.ts

不变量	执行机制	源码位置
Atlas 永不自己实现	`DIRECT_WORK_REMINDER` 注入	system-reminder-templates.ts
每次委派后必须验证	4-Phase verification 在 prompt 中强制	default.ts
中断后自动恢复	`session.idle` → `injectBoulderContinuation`	event-handler.ts
进度通过 checkbox 跟踪	`getPlanProgress()` 解析 `- [ ]` / `- [x]`	storage.ts
子 agent 无状态，notepad 持久化	Atlas prompt 要求 read/append notepad	default.ts

总结

Plan/Build 模式的核心设计哲学是**关注点彻底分离**：

1. **Prometheus 只规划，永不执行** — 通过 `prometheus-md-only` hook 在系统层级强制（非仅 prompt 约束）
2. **Atlas 只编排，永不实现** — 通过 `write-edit-tool-policy` hook 检测越权
3. **Sisyphus-Junior 只执行，不做架构决策** — 通过 task() 的 6 段式 prompt 限定范围
4. **Boulder State 是唯一状态源** — `.sisyphus/boulder.json` 跟踪活跃计划，支持跨会话恢复
5. **Notepad 是累积智慧** — 子 agent 无状态，所有发现持久化到 `.sisyphus/notepads/`

这不是简单的 "先规划后执行"；而是一个**多层守卫 + 自动恢复 + 并行编排**的完整工程系统。